

Frente 18 - Relatorio Completo de Execucao ate 18.6.P1.5

Base deste relatorio:

- ? diagnostico fechado em 18.2.8, 18.2.9, 18.3, 18.4.3, 18.5.3 e 18.6
- ? execucoes fechadas em 18.6.P0.2, 18.6.P0.4, 18.6.P0.5, 18.6.P0.6, 18.6.P1.1, 18.6.P1.2, 18.6.P1.3, 18.6.P1.4 e 18.6.P1.5

Este documento consolida a Frente 18 em uma unica leitura: o que foi fechado como diagnostico, o que virou regra operacional, o que foi implementado em codigo, o que foi validado, o que ficou deliberadamente adiado e qual e o estado final da frente neste ponto.

1. Escopo do relatorio

Entram neste relatorio:

- ? fechamento estrutural da Frente 18 como diagnostico + plano de acao
- ? execucoes realizadas em codigo e documentacao ate 18.6.P1.5
- ? validacoes objetivas registradas nas etapas
- ? divida remanescente explicitamente adiada

Ficam fora:

- ? frentes fora da 18, exceto quando ja foram referenciadas como base fixa
- ? redesign amplo de dominio, banco ou infraestrutura nao executado
- ? remocao final de legado ainda isolado, mas nao aposentado

2. Resumo executivo

A Frente 18 saiu do modo de investigacao e foi encerrada como base de execucao. O diagnostico estrutural, a fila objetiva de acao, a matriz operacional do banco, o plano de queries/indices e a politica de governanca foram consolidados em uma visao unica e operacional.

Depois desse fechamento, a frente avancou em nove entregas de execucao. O sistema deixou de tratar varias ambiguidades como detalhes cosmeticos e passou a separar explicitamente owner canonico, snapshot, compat residual, derivado perigoso e morte futura. Isso apareceu em codigo, em contratos HTTP, em trilha de banco, em fluxo de storage/documentos, em tuning local de queries, em regra de restore e em isolamento de artefatos operacionais residuais.

O resultado pratico ate aqui e:

- ? fonte de verdade por eixo declarada
- ? backlog curto separado por tipo de trabalho
- ? primeira camada de correcao estrutural aplicada
- ? compat residual isolado em vez de invisivel
- ? tuning local feito sem mascarar divida estrutural

? trilha operacional mais previsível para banco, restore e scripts de manutenção

3. O que foi fechado como diagnóstico

3.1 Matriz estrutural de schema

Foi fechado que:

- ? `tripulantes.status` e `tripulantes.base` são colunas ambíguas e competem com `owner` mais adequado em pilotos
- ? `treinamentos.aeronave_modelo` é ambígua no modelo atual e não pode continuar sustentando semântica de programa como se fosse `owner`
- ? `foto_base64`, `arquivo_pdf` e `db:bytea` são compat residual, não trilha principal
- ? `tripulantes.possui_foto` e `periodicidade_meses_tipo` são derivadas perigosas/toleradas
- ? a completude estrutural de `treinamentos` é dívida estrutural ativa

3.2 Fila objetiva de ação

Foi fechado que a frente trabalha em quatro trilhas:

- ? corrigir agora
- ? isolar
- ? congelar
- ? aposentar depois

Com isso, cada item passou a ter:

- ? `owner` esperado
- ? papel operacional
- ? compat residual explícito, quando existe
- ? condição futura de saída

3.3 Matriz operacional do banco

Foi fechado que a evolução do banco não pode continuar misturando:

- ? bootstrap estrutural
- ? migrações corretivas
- ? seeds históricos
- ? syncs operacionais
- ? repairs manuais

Com isso, scripts como `migrations.py`, `training_program_seed.py`, `sync_training_master_types.py`, `cleanup_operational_data.py`, `sync_tripulantes_snapshot.py` e `run_db_consistency.py --repair` deixaram de ser tratados como parte normal da trilha estrutural.

3.4 Queries, índices e governança

Foi fechado que:

- ? nem todo gargalo e falta de indice
- ? tuning local precisa ser separado de divida estrutural
- ? indice em aeronave_modelo depende de SQL real, seletividade real e EXPLAIN
- ? cache, snapshot legitimo, snapshot perigoso, compat residual e restore canonico precisam ter politica operacional explicita

3.5 Fechamento integrado da frente

Foi fechado em 18.6 que a Frente 18 pode ser considerada encerrada como diagnostico + plano de acao quando:

- ? a fonte de verdade por eixo esta declarada
- ? compat historico esta separado do modelo atual
- ? o backlog esta curto, classificado e executavel
- ? a discussao deixa de ser "o que e o problema" e passa a ser "em que ordem executar"

4. O que foi executado em codigo e operacao

4.1 18.6.P0.2 - Owner semantico de treinamentos.aeronave_modelo

Decisao implementada:

- ? treinamentos.aeronave_modelo foi rebaixado a snapshot do treinamento realizado
- ? o owner canonico do dominio continuou em horas_voo_aeronave
- ? a aplicacao e os contratos passaram a distinguir:
- ? aeronave_modelo_referencia
- ? aeronave_modelo_snapshot

Mudancas aplicadas:

- ? fluxo de programa passou a consumir referencia canonica e persistir snapshot
- ? listagem e detalhe de registros por programa passaram a expor snapshot explicitamente
- ? API generica de treinamentos passou a expor origem do registro e snapshot quando o treinamento vem de programa
- ? fluxo generico deixou de aceitar escrita programatica ambigua

Impacto:

- ? aeronave_modelo deixou de sustentar regra futura como owner implicito
- ? o sistema passou a distinguir referencia canonica de programa e historico do realizado

Validacao registrada:

- ? 20 testes executados
- ? 20 testes aprovados

4.2 18.6.P0.4 - Separacao da trilha de migrations

Decisao implementada:

- ? bootstrap estrutural passou a ter trilha propria
- ? `migrations.py` foi rebaixado para corretivas/compat
- ? `seed historico`, `sync`, `import` e `repair amplo` saíram da trilha principal

Mudancas aplicadas:

- ? criacao de comando canonico de bootstrap estrutural
- ? separacao entre `execute_schema_bootstrap`, `execute_migrations` e `seed operacional`
- ? documentacao operacional ajustada para refletir bootstrap, `consistency` e `repair` como caminhos distintos

Impacto:

- ? evolucao do banco ficou menos imprevisivel
- ? operador deixou de confundir bootstrap com `seed historico`, `import` ou `repair`

Validacao registrada:

- ? testes cobrindo separacao entre bootstrap, `repair` e `seed historico`
- ? confirmacao de que `training_program_seed` saiu da trilha estrutural principal

4.3 18.6.P0.5 - Congelar novas escritas em `foto_base64` e `arquivo_pdf`

Decisao implementada:

- ? novas escritas de foto passaram a ir para `foto_storage_ref`
- ? novas escritas documentais passaram a ir para `storage_ref`
- ? `foto_base64` e `arquivo_pdf` foram mantidos apenas como leitura de compat

Mudancas aplicadas:

- ? `writers` vivos de foto deixaram de mandar `payload` para `foto_base64`
- ? repositórios de documentos/anexos passaram a bloquear persistencia nova em `db:bytea`
- ? leitura legado continuou aceita, mas com marcacao residual em contrato e warning operacional

Impacto:

- ? blob legado deixou de crescer nos fluxos principais
- ? o legado continua funcionando, mas deixou de parecer caminho canonico

Validacao registrada:

- ? novas fotos nao alimentam mais `foto_base64`
- ? novos PDFs/anexos nao alimentam mais `arquivo_pdf`
- ? leitura fallback continua funcionando para `foto_base64` e `db:bytea`

4.4 18.6.P0.6 - Correcao de caminho quente de query

Escopo executado:

- ? fetch_tripulante_list_page
- ? count_treinamentos
- ? fetch_training_page
- ? notificacoes

Mudancas aplicadas:

- ? listagem de tripulantes deixou de projetar blob/base64 bruto
- ? contagem de treinamentos deixou de pagar joins de lookup desnecessarios
- ? pagina de treinamentos passou a paginar antes do join mais largo
- ? notificacoes passaram a limitar horizonte efetivo, calcular gatilho no SQL e evitar segunda leitura obrigatoria do mapa de envios

Impacto:

- ? custo local das consultas quentes caiu por simplificacao de query e shape da leitura
- ? a etapa tratou query/paginacao/compat residual sem fingir que tudo era problema de indice

Limitacao registrada:

- ? EXPLAIN real ficou bloqueado por falta de credencial valida do banco local

4.5 18.6.P1.1 - Revisao de fetch_training_program_record_list

Mudancas aplicadas:

- ? remocao de GROUP BY e de COUNT(a.id) direto na listagem
- ? remocao de joins amplos com anexos e horas_voo_aeronave
- ? uso de subconsulta correlacionada para total de anexos
- ? uso de EXISTS para ctac_required

Impacto:

- ? o hotspot deixou de inflar a lista por fan-out de join e agregacao
- ? a listagem passou a carregar apenas o necessario para a tela principal

Limitacao registrada:

- ? EXPLAIN real continuou bloqueado por falta de DATABASE_URL funcional

4.6 18.6.P1.2 - Revisao de build_habilitacoes_consolidadas_context

Mudancas aplicadas:

- ? separacao entre query de tripulantes e query de treinamentos
- ? remocao de ORDER BY desnecessario no banco
- ? centralizacao dos filtros na trilha correta
- ? remocao de status_class do contexto base, preservando-o apenas no adaptador legado

Impacto:

- ? a etapa separou custo de query de custo de montagem
- ? o contexto deixou de recalcular mais do que o necessario
- ? a ordenacao final ficou apenas na etapa em que o payload realmente e montado

Limitacao registrada:

- ? EXPLAIN real nao foi obtido por indisponibilidade de credencial valida

4.7 18.6.P1.3 - Indice de aeronave_modelo so com evidencia

Decisao executada:

- ? nao criar indice novo nesta etapa
- ? nao criar indice funcional em LOWER(COALESCE(t.aeronave_modelo, ''))
- ? nao tratar o indice bruto existente como resposta automatica ao hotspot

Motivo:

- ? o predicado quente esta contaminado por funcao sobre a coluna
- ? a seletividade operacional nao foi provada em banco real
- ? EXPLAIN real nao ficou disponivel

Impacto:

- ? a frente evitou tuning cego
- ? o problema de aeronave_modelo continuou corretamente classificado como sensivel a semantica e medicao real

4.8 18.6.P1.4 - Restore canonico

Decisao implementada:

- ? restore canonico passou a exigir:
- ? dump
- ? assets
- ? config
- ? manifest
- ? validacao pos-restore

Mudancas aplicadas:

- ? formalizacao da diferenca entre restore parcial, restore auxiliar e restore canonico
- ? validacao objetiva de metadata/blob no ambiente restaurado
- ? docs operacionais ajustadas para descrever pre-condicoes, ordem, checagens e criterio de sucesso

Impacto:

- ? banco e storage deixaram de ser tratados como superficies independentes no restore
- ? restore deixou de ser "subir dump e torcer"

4.9 18.6.P1.5 - Aposentar/isolar artefatos operacionais residuais

Escopo executado:

- ? sync_tripulantes_snapshot.py
- ? cleanup_operational_data.py
- ? run_db_consistency.py --repair
- ? novos usos de sistema_controle

Mudancas aplicadas:

- ? run_db_consistency.py em maintenance passou a validar apenas; repair saiu para manual_unsafe
- ? cleanup destrutivo passou a existir apenas em trilha manual/perigosa
- ? sync de tripulantes passou a existir apenas como entrada residual em compat_residual
- ? sistema_controle ganhou policy explicita e bloqueio de novo uso fora da allowlist atual
- ? release gate deixou de aceitar repair embutido como fluxo normal

Impacto:

- ? artefatos residuais deixaram de parecer parte da evolucao principal do sistema
- ? o risco operacional caiu porque repair, cleanup e snapshot residual ganharam barreiras explicitas
- ? sistema_controle deixou de ser superficie aberta para novo estado generico

Validacao registrada:

- ? 17 testes direcionados executados com sucesso
- ? maintenance bloqueando --repair
- ? snapshot residual exigindo ack explicito
- ? cleanup exigindo confirmacao explicita

5. Consolidacao das validacoes

Validado com evidencia forte:

- ? fechamento semantico de aeronave_modelo com testes de unidade e contrato
- ? separacao de bootstrap, migration corretiva e seed historico
- ? congelamento de novas escritas em blob legado mantendo leitura de compat
- ? tuning local dos caminhos quentes com SQL capturado e testes de shape/contrato
- ? revisao local de fetch_training_program_record_list
- ? revisao local de build_habilitacoes_consolidadas_context
- ? formalizacao de restore canonico
- ? isolamento operacional de repair, cleanup, snapshot residual e novos usos de sistema_controle

Validado apenas parcialmente:

- ? etapas de performance que dependiam de EXPLAIN/EXPLAIN ANALYZE real em banco local

Motivo da validacao parcial:

? o ambiente de execucao nao expunha credencial valida para o Postgres local

Nao foi provado nesta frente:

? ganho quantitativo final de plano real em banco vivo para todos os hotspots revisados

? aposentadoria fisica final dos artefatos residuais ainda preservados por compatibilidade controlada

6. O que ficou pendente

Permanece pendente:

? owner semantico e condicao de saida de `tripulantes.status`

? owner semantico e condicao de saida de `tripulantes.base`

? completude estrutural mais ampla de treinamentos

? medicao com EXPLAIN real para queries revisadas em ambiente com credencial valida

? decisao futura de indice funcional ou reescrita adicional em predicados de `aeronave.modelo`, so depois da medicao real

? aposentadoria final de `foto_base64`, `arquivo_pdf`, `db:bytea`, `sync_tripulantes_snapshot.py`, `run_db_repair.py` e `cleanup_operational_data.py`

? retirada futura de `sistema_controle` como superficie mista quando notificacoes e cache residual tiverem owner proprio

7. Impacto consolidado da janela total da Frente 18

Do ponto de vista estrutural:

? a frente deixou clara a fonte de verdade por eixo

? compat historico parou de competir silenciosamente com owner canonico

? backlog ficou separado por divida estrutural, tuning local, compat residual e governanca

Do ponto de vista de codigo:

? ambiguidades centrais viraram contrato executado

? fluxos vivos deixaram de ampliar legado

? consultas quentes ganharam reducao de custo local sem tuning ornamental

? banco, restore e scripts operacionais ficaram mais previsiveis

Do ponto de vista operacional:

? bootstrap, consistency, repair, sync, cleanup e compat residual deixaram de parecer a mesma trilha

? restore passou a ter criterio claro de sucesso

? compat residual ficou observavel, nao invisivel

8. Proximos passos naturais

Sequencia mais coerente a partir daqui:

1. fechar tripulantes.status
2. fechar tripulantes.base
3. atacar a completude estrutural restante de treinamentos
4. medir em banco real os hotspots ja revisados para decidir o que e query, o que e indice e o que e custo legitimo
5. converter repairs genericos e compat residual remanescente em trilhas de saida definitivas

9. Fechamento final

Estado atual da Frente 18 neste ponto:

- ? fechada como diagnostico
- ? fechada como plano de acao
- ? avancada em execucao real
- ? com backlog residual objetivo
- ? sem necessidade de reabrir investigacao base

Declaracao curta:

- ? a Frente 18 deixou de ser um problema difuso e virou uma fila executavel, com parte relevante ja implementada, parte validada e parte deliberadamente adiada com condicao de saida explicita